

Amazon EC2 (Elastic Compute Cloud) - Service Overview

Introduction

Amazon Elastic Compute Cloud (EC2) is a web service that provides resizable compute capacity in the AWS cloud. It is designed to make web-scale cloud computing easier for developers by removing the need to invest in physical hardware. EC2 instances are virtual servers that run in Amazon's data centers, and customers pay only for the compute capacity they actually use.

Instance Types

EC2 offers instance families optimized for different workloads:

- General purpose (T3, T4g, M5, M6i): balanced compute, memory, and networking.
- Compute optimized (C5, C6g): high-performance processors, ideal for batch processing and media transcoding.
- Memory optimized (R5, X2gd): designed for memory-intensive applications such as in-memory databases.
- Storage optimized (I3, D3): high sequential read/write access to large datasets on local storage.
- Accelerated computing (P4, G5): GPU or FPGA instances used for machine learning and graphics workloads.

Pricing Models

EC2 supports multiple purchasing options:

- On-Demand: pay by the second with no long-term commitment, ideal for unpredictable workloads.
- Reserved Instances: commit to a 1-year or 3-year term for a significant discount (up to 72%) versus On-Demand pricing.
- Spot Instances: bid on spare EC2 capacity at discounts of up to 90%, suitable for fault-tolerant, flexible workloads. AWS can reclaim Spot capacity with a two-minute warning.
- Savings Plans: flexible pricing model offering lower prices in exchange for a commitment to a consistent amount of usage (measured in \$/hour) over a 1 or 3 year term.

Elastic IP Interaction

EC2 instances receive a dynamic public IP address by default, which changes when the instance is stopped and restarted. Customers who need a static, persistent public IP address associate an Elastic IP address with their instance. Unlike the default public IP, an Elastic IP remains allocated to the account until explicitly released, and is not tied to a specific instance's lifecycle.

Storage

EC2 instances use Elastic Block Store (EBS) volumes for persistent block storage, or instance store volumes for temporary storage that is lost when the instance stops. Root volumes are typically EBS-backed, allowing the instance to be stopped and restarted without data loss.

Security Groups

Security Groups act as a virtual firewall controlling inbound and outbound traffic at the instance level. Rules are stateful: if you allow inbound traffic on a port, the corresponding outbound response traffic is automatically allowed regardless of outbound rules.

Auto Scaling Integration

EC2 instances can be placed in an Auto Scaling Group to automatically add or remove instances based on demand, ensuring the application maintains steady, predictable performance at the lowest possible cost.

Pricing Example

Consider an m5.large instance (2 vCPU, 8 GiB RAM) running On-Demand in us-east-1 at approximately \$0.096 per hour. Running continuously for a month (730 hours) costs roughly \$70. The same instance under a 1-year No Upfront Reserved Instance commitment drops to roughly \$0.061 per hour (about \$44.50/month), a 36% saving. If the workload can tolerate interruption, a Spot Instance for the same type might cost \$0.03-0.04 per hour, but AWS can reclaim the capacity with only a two-minute warning, so Spot is unsuitable for stateful, always-on services like a primary application server.

EC2 vs ECS vs Lambda

These three compute options sit at different levels of abstraction. EC2 gives you a full virtual machine: you manage the operating system, patching, and scaling yourself. ECS (particularly on Fargate) removes the need to manage servers while still running long-lived containerized processes you control. Lambda goes further, running short-lived functions in response to events with no persistent process at all. A RAG microservice with a long-running FastAPI process handling continuous traffic is a natural fit for EC2 or ECS; a one-off document-ingestion trigger fired when a file lands in S3 is a better fit for Lambda.

Common Use Cases

- Hosting web applications and APIs

- Running batch processing jobs
- Development and test environments
- High-performance computing (HPC) clusters
- Machine learning training and inference

Best Practices

- Right-size instances based on actual CloudWatch utilization metrics rather than over-provisioning.
- Use IAM instance profiles instead of embedding AWS credentials on the instance.
- Enable detailed monitoring for production workloads requiring 1-minute granularity metrics.
- Terminate unused instances promptly, as On-Demand billing continues while an instance is running, even if idle.